

PYTHON ASSIGNMENT 4-

CUSTOMER SUPPORT TICKET ANALYSER SCREENSHOT

Problem Statement

Customer support teams handle numerous service tickets daily. Analysing support tickets helps identify common issues, customer sentiment, support quality, and areas for improvement.

In this project, you will build a **Python-based Ticket Analysis System** that stores, cleans, analyses, and extracts insights from customer support tickets.

Step 1: Preloaded Tickets

Begin your program with the following dictionary of lists containing 10 customer tickets:

Screenshot:

Output:

```
{'ticket_num': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 'customer_name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', ''],  
Ticket No: 1  
Customer Name: Ravi  
Issue: internet not working!!!  
Priority: High  
Ticket No: 2  
Customer Name: Meera  
Issue: slow response, very poor service  
Priority: Low  
Ticket No: 3  
Customer Name: Sam  
Issue: GREAT support! issue resolved  
Priority: High  
Ticket No: 4  
Customer Name: Anu  
Issue: okay... need help  
Priority: Medium  
Ticket No: 5  
Customer Name: Rakesh  
Issue: not BAD but slow  
Priority: Low  
Ticket No: 6  
Customer Name: Divya  
Issue: Excellent guidance, Very Helpful!  
Priority: High  
Ticket No: 7  
Customer Name: Arjun  
Issue: good support and good behaviour!  
Priority: Medium  
Ticket No: 8  
Customer Name: Kiran  
Issue: Poor handling of technical issue  
Priority: High
```

```
Ticket No: 9
Customer Name: Leela
Issue: Satisfied. Could be better
Priority: Low
Ticket No: 10
Customer Name: Nisha
Issue: Good service...quick response
Priority: Medium
```

- **Task: Print the initial ticket data in a readable format.**

Step 2: Add More Tickets

❖ *Ask the user: How many new tickets do you want to add?*

- ❖ For each new ticket, collect: ➤ Customer Name

➤ Issue Description

➤ Priority (High / Medium / Low)

Screenshot:

Output:

Step 2: Add More Tickets

```

1 ticket_count = int(input("How many new tickets? "))
2
3 for _ in range(ticket_count):
4     name = input("Customer Name: ")
5     issue = input("Issue: ")
6     priority = input("Priority (High/Medium/Low): ").capitalize()
7     new_id = max(ticket_data["ticket_num"]) + 1
8     ticket_data["ticket_num"].append(new_id)
9     ticket_data["customer_name"].append(name)
10    ticket_data["issue_description"].append(issue)
11    ticket_data["priority"].append(priority)
12    print(ticket_data)

```

```

** How many new tickets? 5
Customer Name: Gracy
Issue: printing issue
Priority (High/Medium/Low): high
{'ticket_num': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11], 'customer_name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha', 'Girish']}
Customer Name: cristo
Issue: need help
Priority (High/Medium/Low): low
{'ticket_num': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12], 'customer_name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha', 'Girish']}
Customer Name: jude
Issue: need help
Priority (High/Medium/Low): high
{'ticket_num': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13], 'customer_name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha', 'Girish']}
Customer Name: riya
Issue: need help
Priority (High/Medium/Low): medium
{'ticket_num': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14], 'customer_name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha', 'Girish']}
Customer Name: mary
Issue: need help
Priority (High/Medium/Low): medium
{'ticket_num': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15], 'customer_name': ['Ravi', 'Meera', 'Sam', 'Anu', 'Rakesh', 'Divya', 'Arjun', 'Kiran', 'Leela', 'Nisha', 'Girish']}

```

Screenshot:

Output:

[illegible]

Create a function:

Function Requirements:

- Use this function to print:

- Number of tickets containing "poor"
- Number of tickets containing "good"
- Number of tickets containing "slow"
- Number of tickets containing "excellent"

Screenshot:

Output:

Step 4: Keyword-Based Issue Insights

s

```
1 def count_tickets_with_word(word):
2     count = 0
3     for issue in ticket_data["issue_description"]:
4         if word.lower() in issue.lower():
5             count += 1
6     return count
7
8 print("Poor:", count_tickets_with_word("poor"))
9 print("Good:", count_tickets_with_word("good"))
10 print("Slow:", count_tickets_with_word("slow"))
11 print("Excellent:", count_tickets_with_word("excellent"))
```

```
Poor: 2
Good: 2
Slow: 2
Excellent: 1
```

Step 5: Final Summary & Insights

Produce detailed analytics:

5(1). Display Final Cleaned ticket_data

Nicely formatted dictionary-of-lists output.

Screenshot:

Output:

Step 5: Final Summary & Insights

5(1).Display Final Cleaned ticket_data

```
1 for i in range(len(ticket_data["ticket_num"])):  
2     print("Ticket No:", ticket_data["ticket_num"][i])  
3     print("Customer Name:", ticket_data["customer_name"][i])  
4     print("Issue:", ticket_data["issue_description"][i])  
5     print("Priority:", ticket_data["priority"][i])  
6
```

```
Ticket No: 1  
Customer Name: Ravi  
Issue: internet not working!!!  
Priority: High  
Ticket No: 2  
Customer Name: Meera  
Issue: slow response very poor service  
Priority: Low  
Ticket No: 3  
Customer Name: Sam  
Issue: great support! issue resolved  
Priority: High  
Ticket No: 4  
Customer Name: Anu  
Issue: okayay need help  
Priority: Medium  
Ticket No: 5  
Customer Name: Rakesh  
Issue: not bad but slow
```

5(2). Priority Analysis

Compute:

- Number of High-priority tickets
- Number of Medium priority tickets
- Number of Low-Priority tickets

Screenshot:

Output:

5(2).Priority Analysis

```
1 high_count = ticket_data["priority"].count("High")
2 medium_count = ticket_data["priority"].count("Medium")
3 low_count = ticket_data["priority"].count("Low")
4
5 print("High-priority tickets:", high_count)
6 print("Medium-priority tickets:", medium_count)
7 print("Low-priority tickets:", low_count)
```

```
High-priority tickets: 6
Medium-priority tickets: 3
Low-priority tickets: 6
```

5(3). Find the Ticket With the Longest Issue Description Based on word count, print:

- Ticket number
- Customer name
- Cleaned issue text
- Word count

Screenshot:

Output:

5(3).Find the Ticket With the Longest Issue Description

```
1 longest_count = 0
2 longest_index = 0
3
4 for i in range(len(ticket_data["issue_description"])):
5     word_count = len(ticket_data["issue_description"][i].split())
6
7     if word_count > longest_count:
8         longest_count = word_count
9         longest_index = i
10
11 print("Ticket number:", ticket_data["ticket_num"][longest_index])
12 print("Customer name:", ticket_data["customer_name"][longest_index])
13 print("Cleaned issue:", ticket_data["issue_description"][longest_index])
14 print("Word count:", longest_count)
```

```
Ticket number: 2
Customer name: Meera
Cleaned issue: slow response very poor service
Word count: 5
```

5(3).Find the Ticket With the Longest Issue Description

```
1 longest_count = 0
2 longest_index = 0
3
4 for i in range(len(ticket_data["issue_description"])):
5     word_count = len(ticket_data["issue_description"][i].split())
6
7     if word_count > longest_count:
8         longest_count = word_count
9         longest_index = i
10
11 print("Ticket number:", ticket_data["ticket_num"][longest_index])
12 print("Customer name:", ticket_data["customer_name"][longest_index])
13 print("Cleaned issue:", ticket_data["issue_description"][longest_index])
14 print("Word count:", longest_count)
```

```
Ticket number: 2
Customer name: Meera
Cleaned issue: slow response very poor service
Word count: 5
```

5(4). Extract Unique Words Used

Generate a set of all unique words across all issue descriptions.

Show:

- Count of unique words
- The word list (sorted)

Screenshot:

Output:

5d.Extract Unique Words Used

```
1 unique_words = set()
2
3 for issue in ticket_data["issue_description"]:
4     words = issue.split()
5     unique_words.update(words)
6
7 sorted_words = sorted(unique_words)
8
9 print("Count of unique words:", len(sorted_words))
10 print("Sorted unique words:")
11 print(sorted_words)
```

```
... Count of unique words: 34
Sorted unique words:
['and', 'bad', 'be', 'behaviour!', 'better', 'but', 'could', 'excellent', 'good', 'great', 'guidance', 'handling', 'help', 'helpful', 'internet', 'issue', 'need', 'not', 'of
```